

RAG TECHNIQUES

Retrieval-Augmented Generation

From Basic RAG to Reranking

LangChain · Ollama · ChromaDB · CrossEncoder



What We'll Cover

01

What is RAG?

The open-book exam analogy

02

Basic RAG

Load → Chunk → Embed → Retrieve → Answer

03

Query Rewriting

Smarter queries → better chunks

04

HyDE

Hypothetical Document Embeddings

05

Multi-Query

Broad coverage via parallel retrieval

06

Reranking

Precision filtering with CrossEncoder

What is RAG?

✗ LLM Without RAG

Closed-book exam

- Answers from training data only
- May hallucinate or give outdated info
- No access to your private documents

✓ LLM With RAG

Open-book exam

- Retrieves relevant facts first
- Grounds the answer in real content
- Works with any private document

Retrieval

+

Augmented

+

Generation

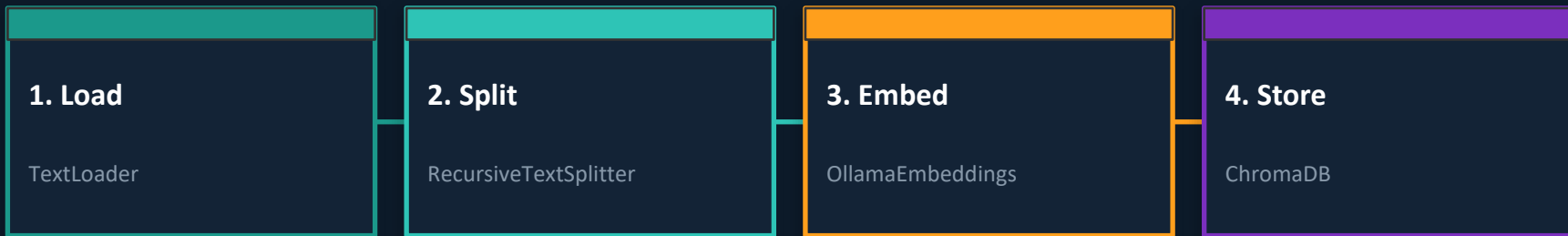
01

Basic RAG

Load · Chunk · Embed · Retrieve · Generate

Basic RAG — The Pipeline

INDEXING PHASE (run once)



↓ Query time

QUERYING PHASE (per question)

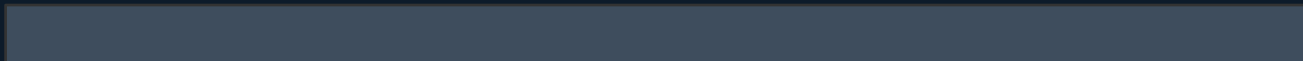


Why Chunk? — The Book Index Analogy



Just like a book index — you don't retrieve the whole chapter, you retrieve the exact page that answers the question.

Full Document



Chunk 1



Chunk 2



chunk_size = 200

→ Max characters per chunk — tune based on document density

chunk_overlap = 50

→ Shared characters between adjacent chunks — prevents losing context at boundaries

Embeddings — Text as Numbers

 Like plotting cities on a map using coordinates — cities that are geographically close appear near each other. Embeddings do the same for meaning.

"Leave policy"

→ [0.12, -0.45, 0.87, ...]

Similar topic →

"Annual leave"

→ [0.14, -0.43, 0.85, ...]

very similar vector 

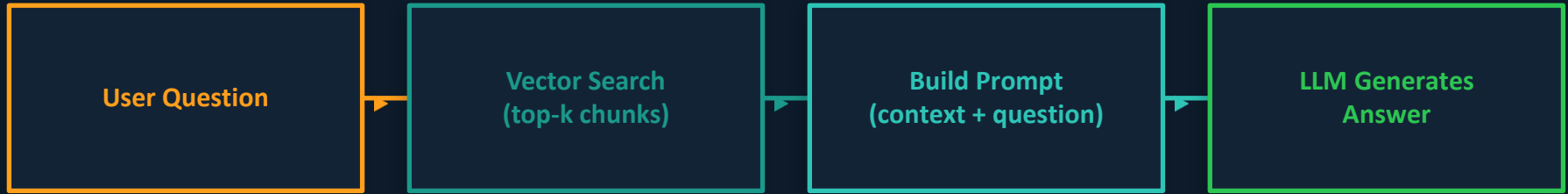
"Server firewall"

→ [0.91, 0.23, -0.10, ...]

Different topic → very different vector 

Model used: nomic-embed-text (runs locally via Ollama — no API key, no data leaving your machine)

Retrieval → Prompt → Answer



Guardrail Prompt

"Answer ONLY using the provided context" — forces the LLM to stay grounded in retrieved chunks, not its training data.

Fallback Message

"I could not find that information in the document" — prevents confident wrong answers on out-of-scope questions.

02

Query Rewriting

Bridge the gap between how users ask and how documents are written

Query Rewriting — The Librarian Analogy



*You say: "Do you have anything about people who sail around the world?"
A good librarian translates that into "circumnavigation + maritime exploration" before searching the catalog.*

Without Rewriting

	Document says	Match?
"Can I take days off?"	"Annual leave entitlement policy"	✗ Poor match
"Who do I tell if sick?"	"Absence notification procedure"	✗ Poor match
"annual leave entitlement"	"Annual leave entitlement policy"	✓ Good match

User Question



LLM Rewrites



Optimised Query



ChromaDB Search



Better Chunks



Answer

How It Works — PromptTemplate & LCEL



PromptTemplate

A reusable, parameterised prompt — like a function where {question} gets filled in at runtime.

Ensures consistent formatting and validates all required variables before sending to the LLM.



LCEL Pipe Operator |

Chains LangChain components left-to-right — output of one step becomes input of the next.

rewrite_prompt | llm
→ Fill template → Send to LLM → Get rewritten query



Just like Unix pipes: `cat file.txt | grep "error"` — output of one step flows directly into the next

03

HyDE

Hypothetical Document Embeddings

HyDE — Search with an Answer, Not a Question

🎵 You don't know the song title — so you hum the tune. HyDE gives the search engine something that sounds like the answer, not just a description of it.

Without HyDE

Question vector:

"What is the leave policy?"



Question-shaped vector

→ semantic mismatch with answer-shaped chunks ❌

With HyDE

Hypothetical answer vector:

"Employees are entitled to 20 days..."



Answer-shaped vector

→ matches answer-shaped chunks much better ✅

HyDE — The 4-Step Flow

1

User Question

Raw, informal question



2

LLM Generates Hypothetical Doc

A fake-but-stylistically-accurate answer paragraph



3

Embed Hypothetical Doc

Answer-shaped vector used for search



4

Retrieve Real Chunks

ChromaDB finds real chunks similar to the hypothetical vector

⚠ Critical Rule: The hypothetical document is a SEARCH PROBE only — it is NEVER shown to the user and NEVER used in the final answer.

04

Multi-Query Retrieval

Search N times, merge results, remove duplicates

Multi-Query — The Expert Researcher Analogy

 A novice searches once and leaves. An expert researcher tries multiple subject headings — synonyms, related terms, different angles — then combines everything.

✗ Problem: A single query only finds chunks semantically close to THAT ONE PHRASING. Relevant info under different terminology gets missed.

"leave policy"

"annual leave entitlement"

"employee absence policy"

"time off rules"

"holiday allowance"

4 queries



4×Retrieve



Merge all



Deduplicate



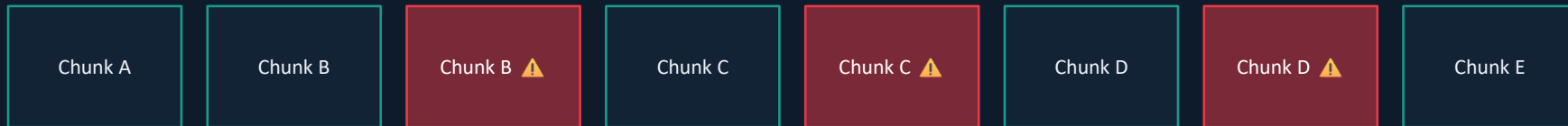
Rich Context



Answer 

Why Deduplication is Non-Negotiable

WITHOUT dedup (4 queries × 2 chunks = 8 raw results)



WITH dedup (5 unique chunks — clean context)



Wastes prompt tokens on repeated content — pushes useful chunks out of context window



Biases the LLM toward the duplicated chunk — it appears more often in the prompt



Drowns out other relevant chunks, reducing answer coverage

05

Reranking

Precision filtering — find the truly relevant chunks

Bi-Encoder vs Cross-Encoder

Bi-Encoder (Retrieval)

nommic-embed-text

- Encodes question and chunk SEPARATELY
- Produces a vector per text
- Vectors pre-computed at index time
- Very fast — milliseconds
- Good accuracy — but similarity $\neq$ relevance

Cross-Encoder (Reranking)

BAAI/bge-reranker-base

- Processes question + chunk TOGETHER as one input
- Outputs a single relevance SCORE (not a vector)
- Must process each pair at query time
- Slower — runs per candidate pair
- Better accuracy — directly models relevance

Strategy: Bi-Encoder retrieves top-5 candidates fast → Cross-Encoder rescores all 5 precisely → Keep top-2

Reranking — The Hiring Process Analogy

Stage 1

HR Keyword Scan

HR scans 500 CVs by keyword matching and shortlists 10 candidates.

Fast, but keyword matching isn't perfect — good CV can be missed.

= Bi-Encoder Retrieval (ChromaDB, k=5)

Stage 2

Hiring Manager Review

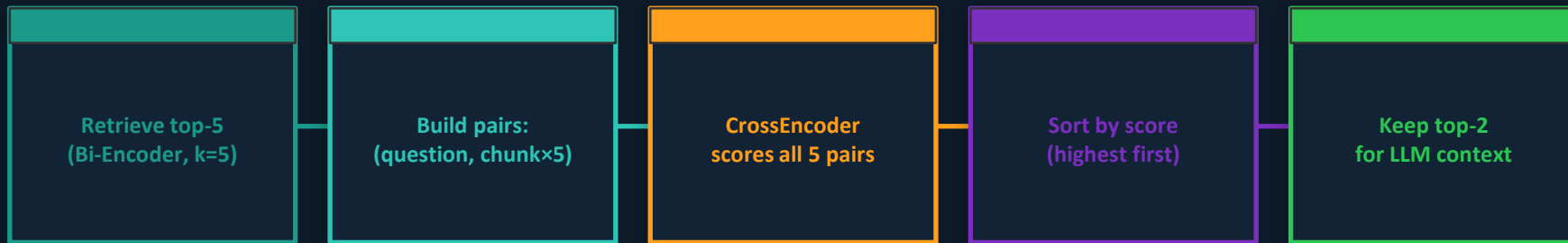
The hiring manager reads those 10 CVs carefully and selects top 2 for interview.

Slower, but far more accurate — reads full context, not just keywords.

= Cross-Encoder Reranking (keep top-2)

💡 You wouldn't ask the hiring manager to read all 500 CVs — that's why you need BOTH stages.

Reranking — Step by Step



Example CrossEncoder Scores:

Chunk 1 +8.42 → Rank 1 ✓	Chunk 2 -1.20 → Not relevant ✗	Chunk 3 +6.71 → Rank 2 ✓	Chunk 4 +2.10 → Rank 3 (dropped)	Chunk 5 -3.55 → Not relevant ✗
---------------------------------------	---	---------------------------------------	---	---



The k=5 rule:

Always over-retrieve before reranking. If k=2 and you keep top-2, there's nothing to rerank. Rule of thumb: retrieve 3×–5× more than you want to keep.



Negative scores are normal — CrossEncoder output is unbounded. Negative = the chunk is not relevant to this question.

All 5 Techniques — Side by Side

Technique	Search Strategy	Strength	Tradeoff
Basic RAG	1 query → top-K	Simple & fast	Term mismatch
Query Rewriting	Rewritten query → top-K	Better vocabulary alignment	+1 LLM call
HyDE	Hypothetical answer → top-K	Best semantic alignment	+1 LLM call
Multi-Query	4 variants → merge → dedup	Broadest coverage	+4 LLM / retrieval calls
Reranking	top-5 → CrossEncoder → top-2	Highest precision	Slower (CrossEncoder)

The RAG Evolution — One Diagram

Basic RAG

Question → Retriever → LLM → Answer

Query Rewriting

Question → LLM rewrites → Retriever → LLM → Answer

HyDE

Question → LLM writes hypothetical answer → Retriever → LLM → Answer

Multi-Query

Question → LLM × 4 variants → 4× Retrieve → Dedup → LLM → Answer

Reranking

Question → Retriever (k=5) → CrossEncoder → Top-2 → LLM → Answer

★ **Power Combo: Multi-Query + Reranking = Broadest Coverage + Highest Precision**

When to Use Which Technique?

Basic RAG

Use when: Prototyping, small doc sets, consistent terminology

Query Rewriting

Use when: Users ask casually / informal language; simple keyword mismatch

HyDE

Use when: Abstract or complex questions; large formal/policy documents

Multi-Query

Use when: Inconsistent doc terminology; multi-faceted or ambiguous questions

Reranking

Use when: High-precision Q&A; latency acceptable; false positives costly

What's Next?



ConversationalRetrievalChain

Add memory so users can ask follow-up questions without repeating context



Multi-Document RAG

Scale to hundreds of files using DirectoryLoader and metadata filtering



RAGAS Evaluation Framework

Measure faithfulness, context recall, and answer relevancy across all 5 techniques



Production Optimisations

GPU CrossEncoder, async retrieval, caching, streaming LLM responses



Hybrid Search (BM25 + Vector)

Combine keyword search with semantic search — best of both worlds



LangChain Agents + RAG

Let an agent decide when to retrieve and which document store to query

Key Takeaway

RAG is not one technique — it's a family of patterns.

Start with Basic RAG. Add Query Rewriting or HyDE when retrieval misses.
Add Multi-Query for broader coverage. Add Reranking for precision.

LangChain · Ollama · ChromaDB · sentence-transformers

Basic RAG

Query Rewriting

HyDE

Multi-Query

Reranking